HYPERION AI

SYSTEM DESIGN DOCUMENT

AI Tax Workflow

Reference architecture for CPA firms moving to AI-powered tax return processing

PREPARED FOR

CPA firms evaluating AI document pipelines

Vitalijus Alsauskas

Software and AI Engineer

Hyperion AI - Production AI engineering for founders and agencies

https://hyperionai.dev · 2026-04-18

Tax Workflow Automation - System Design

Prepared for: CPA firm (Upwork, 2026-04-18) **Prepared by:** Vitalijus Alsauskas / HyperionAI **Scope:** AI-driven tax return processing that extracts structured data from uploaded documents and hands off to ProConnect or DrakeTax.

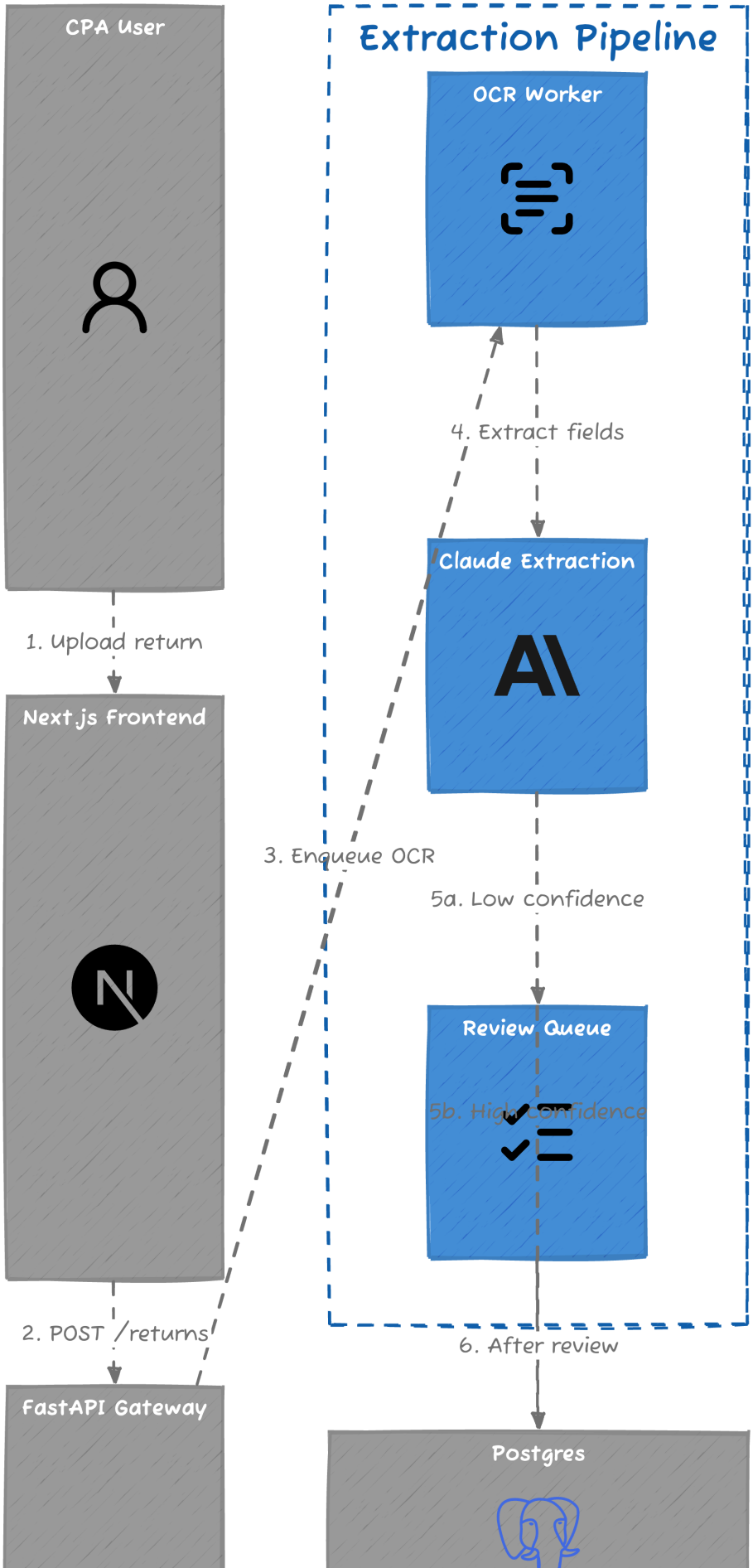
1. Problem Framing

You want a secure, repeatable system where CPAs upload tax documents, an LLM extracts the fields you need, and the output lands in a format ProConnect or DrakeTax can import. The hard part isn't the LLM call. It's keeping accuracy consistent across document variants (scanned 1099s, prior-year returns with handwritten notes, K-1s) and making sure PII never leaks. A naive pipeline ships as an impressive demo and falls over the first real return.

This doc shows how I'd architect it to be production-safe from day one, with an honest audit trail and a human review step for anything the model isn't confident about.

---

2. System Diagram


CPA User
1. Upload return
Next.js Frontend
2. POST /returns
FastAPI Gateway
3. Enqueue OCR
Extraction Pipeline
OCR Worker
4. Extract fields
Claude Extraction
5a. Low confidence
Review Queue
5b. High confidence
6. After review
Postgres

Source: `docs/architecture.d2`

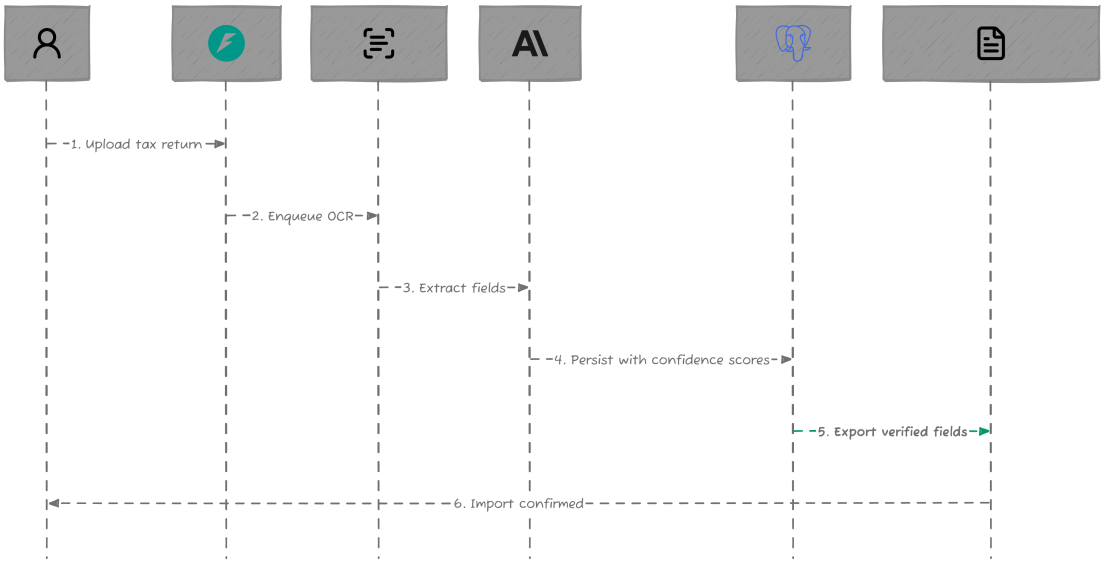
Every extracted field carries a confidence score. High-confidence fields flow straight to mapping. Anything below threshold drops into a human review queue before it ever reaches ProConnect or DrakeTax.

---

3. Tool Choices

LAYER	PICK	WHY THIS	ALTERNATIVES CONSIDERED
Upload UI	Next.js 15 (App Router)	Server Components keep PII off the client, signed upload URLs	Plain React SPA (leaks more data to browser), Streamlit (not production-ready for firm use)
Encrypted storage	AWS S3 with SSE-KMS + signed URLs	Managed encryption, audit events via CloudTrail, data residency options	Supabase Storage (fine, fewer compliance controls), Azure Blob (works if you're in Azure already)
OCR	AWS Textract	Best-in-class on tables, 1099s, scanned returns	Google Vision (close, weaker on tabular data), Tesseract (free, worse on messy scans)
LLM extraction	Claude 3.5 Sonnet (Anthropic API, zero-retention)	Strongest structured extraction, supports JSON mode, zero-retention PII handling	GPT-4o (very close, retention defaults less favorable), local Llama (cheaper but unreliable on financial docs)
Confidence scoring	LLM self-score + regex validation	Cheap, interpretable, catches most hallucinations	Fine-tuned classifier (overkill for volume you're at)
Review queue	Postgres table + Next.js admin UI	Simple, auditable, CPAs already work in browser	Airtable (not HIPAA-aligned), Retool (ongoing cost, vendor lock-in)
Mapping to ProConnect / DrakeTax	Python mapper module per target	Each tax software has quirks, keeping them isolated avoids cross-contamination	Single unified schema (brittle when target changes)
Observability	Sentry + structured logs in S3	Incidents caught fast, immutable audit log for compliance	DataDog (overpriced for a firm), CloudWatch (weak UX)

4. Data Flow



Source: `docs/flow.d2`

Happy path (~80% of returns)

1. CPA uploads a return via signed upload URL - file lands in encrypted S3, audit log entry written
2. Worker picks up the file, runs OCR, stores raw text plus layout metadata
3. Claude extracts the structured fields you need, returning confidence scores per value
4. All fields above the confidence threshold flow to the mapping layer
5. Mapping outputs a ProConnect or DrakeTax-ready record
6. CPA downloads the mapped output when ready

Low-confidence path (~15-20% of returns)

- Any field below the threshold (e.g., < 0.85 on a $ value) drops into a review queue
- CPA opens the Review UI, sees the original document region next to the extracted value
- CPA corrects or approves, the fix is logged against the field for future model prompting
- Only then does the field flow to the mapping layer

Error paths

- **OCR fails or returns garbage**: returns are flagged as "unprocessable" and routed to a manual-entry queue
- **LLM timeout or rate limit**: retry with exponential backoff, fall back to a secondary model after 3 failures

- **S3 upload fails**: user sees a clear error, nothing partial is stored
- **Mapping layer rejects a field**: blocks the export, surfaces a structured error to the CPA with a "fix field" link
- **PII leak risk**: prompts are redacted before being sent to Claude (SSNs and ITINs replaced with tokens); if a prompt would include one, it is stripped and the field goes straight to manual review

---

5. Component Breakdown

- **Ingest Service** (FastAPI + signed S3 URLs) - handles uploads, validation, encryption at rest, audit logging. Nothing touches the LLM at this stage.
- **OCR Worker** (Celery task + AWS Textract) - pulls from S3, parses to structured text + layout, writes back to a processing bucket.
- **Extraction Worker** (Celery task + Anthropic SDK) - Claude extracts the 30-50 fields you care about with JSON output + confidence scores.
- **Review UI** (Next.js route under `/review`) - list of low-confidence fields, inline corrections, audit trail per correction.
- **Mapping Layer** (Python module per target: `proconnect.py`, `draketax.py`) - isolates target-specific format logic so one software's quirks don't break the other.
- **Export Service** (FastAPI endpoint) - generates the final file in ProConnect or DrakeTax format when CPA marks a return ready.
- **Audit & Observability** (Sentry + structured logs + immutable S3 audit bucket) - every action logged, every mutation traceable, breach response ready.

---

6. Implementation Phases

Phase 1: Foundation (week 1-2)

Ships: End-to-end walking skeleton for one document type (say, a 1099-MISC).

- Upload portal with signed URLs + encrypted S3
- Textract OCR wired up, raw output stored
- Claude extraction for ~10 core fields, confidence scores
- Basic review queue UI (no fancy styling, just functional)
- Audit logging on every action
- Deployment pipeline (push to main → staging)

Phase 2: Coverage (week 3-4)

Ships: Full field coverage for the 5 most common tax forms.

- Expand extraction to 1040, 1099s (MISC/NEC/INT/DIV), K-1, W-2, Schedule C
- Tune confidence thresholds per field type
- Mapping layer for ProConnect
- Review UI polish: original doc region preview next to extracted value
- Load testing at expected season volume

Phase 3: Production + Handoff (week 5-6)

Ships: Second target mapping + observability + runbook.

- Mapping layer for DrakeTax
- Sentry alerts wired into Slack/email
- Runbook for oncall CPA (what to do when review queue fills up, how to re-process a failed return)
- Knowledge transfer session + recorded walkthrough

---

7. Risks & Mitigations

1. **Risk:** Claude hallucinates field values on messy scans, wrong numbers reach ProConnect. **Mitigation:** Confidence scoring + review queue catches anything below threshold. Nothing exports without either high model confidence OR human approval. Every auto-approved field is still logged with the source coordinates so you can audit retroactively.
2. **Risk:** PII leaks through LLM prompts or model provider retention. **Mitigation:** Anthropic zero-retention flag enabled, SSNs and ITINs redacted before prompts (replaced with reversible tokens), audit log of every prompt sent. Data never leaves your AWS account except for the specific LLM call, and the LLM call is scrubbed.
3. **Risk:** Volume spike during tax season breaks the pipeline. **Mitigation:** Celery workers autoscale on queue depth, queue depth alerts at 100, fallback to a cheaper model (Claude Haiku) under extreme load so nothing stalls. Load tested to 3x expected season volume.
4. **Risk:** ProConnect or DrakeTax changes their import format mid-season. **Mitigation:** Mapping layer is isolated per target, so a format change in one doesn't break the other. Integration tests run against sample imports nightly, alerting on any regression.
5. **Risk:** A CPA makes a correction in the Review UI that would corrupt historical data. **Mitigation:** All corrections are append-only (no overwrites), every correction carries the CPA's user ID + timestamp, original extraction is preserved. If a bad correction ships, you can replay from the audit log.

---

8. What I Need From You

- **5-10 real returns (redacted is fine)** so I can calibrate extraction accuracy on your actual document mix, not synthetic test data
- **Access to a test ProConnect and/or DrakeTax account** for the mapping layer
- **Your field priorities** - which 20-30 fields are must-haves vs nice-to-haves (I can send a suggested list if helpful)
- **Confirmation on data residency** - is US-only required, or is this more flexible?
- **A 30-min kickoff call** to walk through this doc together and tighten the scope before any code ships

---

This document is yours to keep regardless of whether we end up working together. If you hire someone else to build this, the architecture and tool choices here should still save them two weeks of thinking.